

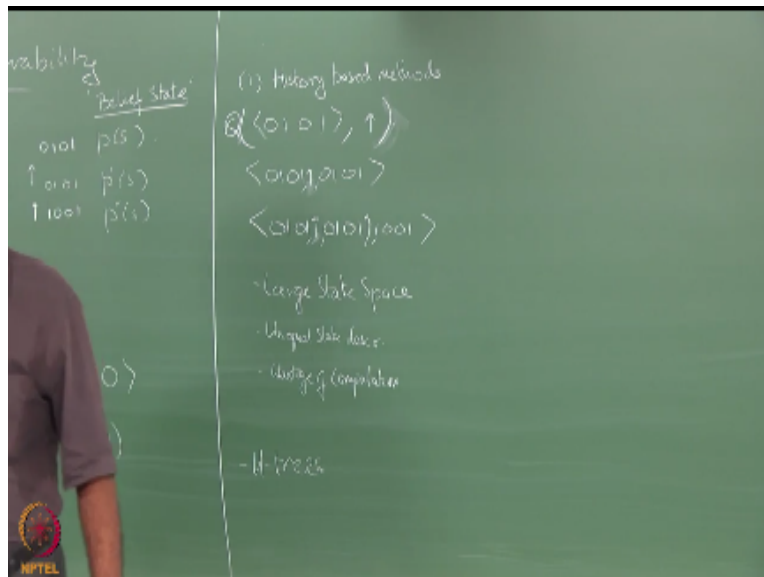
NPTEL
NPTEL ONLINE COURSE

REINFORCEMENT LEARNING

Solving POMDP

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

(Refer Slide Time: 00:16)



So I am going to talk about different ways of thinking about solving this problem right, the first one is actually something that does not use belief states right pretty straightforward pretty very nice way of solving this is basically so I am going to call them history based methods, but almost all the partial observable solution methods use history in some fashion or the other but I call

these history based methods because they use history very directly so what do they do and then keep a track do what with it.

No, I told you right this first method does not use beliefs how else can you use the observations directly. We talked about it we talked about in the options case when I said we talked about semi Markov options what did I say look at the history since the option started or something like that exactly the same thing you look at history forever right, so your state space will start looking really funny your state space will look like the first state is 0101, okay.

So the next state is next state is the next state is for this right so the most recent state will be the last entry here, right so and then you go back in history you remember the whole history like this, right so this is my state. So if I am learning a q function I will learn q of, good point if you want you can use the action as well, so the history can contain just the sequence of observations you have made or it could also have the action if you want you can put the action itself that is a good point so you could also possibly do this, right so you could also have the history as I mean action as part of the history.

So the thing is so 0101 and then 1001 I keep going up right, so that is one thing if I go up i have 0101 then if I go down I have 0101 that is a completely different interpretation if I did two ups and I still get 0101 it is very high probability that 0101 is a noisy measurement if I go up I get 0101 and then I go down I get 0101 that is probably the right thing right probably I went from here to here and then I came back here.

But if I had gone from here to here to here and I still get the same 0101 there is something wrong right, I could have done either this measurement was wrong or this measurement was wrong or could be something even worse my sensors could be stuck at 0101 right I could have been anywhere in the anywhere in the world, right so knowing the actions helps to disambiguate those kinds of things.

So going I just keep doing up down up, down up down and I keep seeing 010101 that is fine, right if I keep doing up, up, up, up, up and I keep saying 0101 0101 then there is something

wrong it is good to have the actions exactly that is a good question what do you think you should do why you want to repeat I mean if you talk about episodic MDPs and things like that right, so next time you start an episode you will start from the beginning right.

It is like keeping track of a game from when you started playing the game, but you have a point I mean why do you want to repeat but so people people see how this is this can be useful right ok what are the problems with this state space is very large right, so large state space yeah yeah that is why it is large state space yeah so if the episode is large the length becomes large anything else. If you are thinking of doing parameterized representations and things like that your states have different lengths not only is the state space large.

But each description is of unequal lengths right, anything else, I will come to that in a minute there is a wastage of computation but states of low probability I mean that is a problem we have with MDPs also right, I mean if you do not visit them often enough you are not going to waste computation resources on them, right. Sorry you could you could maybe I am not sure.

I mean given the way the field is moving I would say you would want to use an LSTM and not a HMM but yeah you could think of sequence models for doing this history modeling that is exactly what I am suggesting to you guys to use talk to him later, right. So one thing that you want to think about so when do you think this will work is it a good idea will it always work. What is it that we require of a state in MDP think of the math what did you say given what is enough we have a specific definition for a Markov-ness right what is the definition.

You guys are all missing a point you are missing an important point so they decided to nominate you as a TA is it, you are missing an important point probability of S_{t+1} given S_t, A_t is equal to exactly, so sufficient enough will determine the next state all of that is fine, but only to the extent of what you could have done with the entire history that is all I am asking for right I do not want the current state to have more information than I could have had by remembering the entire history that is exactly what we are doing here.

If you can remember the entire history you are essentially in an MDP right, so you cannot ask for anything more ok is it fine, is it correct I am just remembering history of the part just the history of the observations right, so here there is a very nice question about the broken vision system in the book I do not think we, did we assign it to you guys think about that now, right. I remember the history from the beginning and the history is 01010101010 is that enough, I mean I can as well be anywhere in the world, right.

So I do not get the state information per say because there is a significant level of noise introduced in the observations, correct so assuming that remembering history is good enough so assuming that I know where I was to start out with and remembering history is good enough then this should be completely sufficient but sometimes it is not, okay. Suppose my observations is such that, right I set up a k -th order Markov system what is a k -th order Markov system it depends only on the k last states, right.

Normally what we talk about Markov is first order, right so k -th order Markov is it depends on the K previous states, right. So if I, if my observations is such that I have a k -th order Markov system so for the last k observations k can be however large you want can be arbitrarily large right, but if there exists a k such that if you remember everything it is Markov then all of this will work okay, that need not necessarily be the case as I just told you in the like in the broken vision system right, but then there is a question that I asked you is the broken vision system Markov or not, right in the textbook and the answer was yes, it is, because there is no underlying state space, right.

So we just assume that that is all the state is right so we did not assume that there was an underlying state space the observations was the state, right and therefore I can predict the next observation very clearly right, it is going to be 0101 duh correct, in the broken vision system in the broken vision system I can always predict what the next observation will be it will be a completely blank screen, right.

I do not need history because it is broken I know that it will not change therefore it is Markov what is somewhere yeah, that is a different issue that is a different what is that, there is not a, it

starts out being broken yeah, yeah but we did not put that into the problem statement did we it is broken it stays broken. What has to be modelled in the MDP. Yeah, so they are not there, it is not mentioned means, it is not there if you are making, you always know that rule, right.

If you are making any assumptions about the problem write the assumptions clearly and then write the answers, so if you are not making any assumptions then but anyway so getting back to this so if your problem is k-th order Markov right history is good, right having history is actually useful thing to have right. So but we still have the problem of unequal state distributions unequal state lengths, right how can we fix that, no no this state representation is unequal.

So there are many ways of fixing it for example, if you use LSTMs you will actually get a single I mean fixed length representation and all that, but one simple way to do it is to fix your history we are going to say that okay, k-th order Markov I am going to look at the last k states and then if I have only seen three states so far to fill it with dummy values, that should be like a no-op kind of a symbol, right that is a maybe 1111 basically I am surrounded all sides by obstacles that can never happen right.

So I can just put 1111 as my symbol right, why well I cannot move right so what is the point, sensors can still report that state yeah, sure, great. Then pick another thing say -1,-1,-1,-1 or whatever, right so pick some sentinel kind of thing right we know for sure it is not a state, right so then what would happen is suppose my history is three right, this guy will be reported as 0101, -1,-1,-1,-1, -1 and this guy will be reported as 0101, 0101, -1 -1 -1 -1 right, this is full state after this everything will be fine.

So there will be a small fraction of the history a small fraction in the beginning which will have this weird -1s but after that everything will be fine okay, that is good. And so we also solved the last start space problems I mean large state space problem right, by using this and what about where is the wastage of computation coming from, I am talking about wastage of computation it still is there. That is one part of it anything else.

For example let us go back to the noise free scenario, if I have a trajectory of 10 states right, and let us say state number 10 is where I am now and state number 9 my observation was 1001, so do I need to remember 1 to 8 right, because 1001 is absolutely localizing right, so I know as soon as I see 1001 I know I do not need the history before that so as soon as I am able to localize myself perfectly the history before that becomes irrelevant right.

So localizing yourself perfectly is just one example there could be other cases where some places you need a history of length 3 some places you need a history of length 2, some places you need a history of length 7 and so on and so forth. It might not be uniformly k -th order Markov throughout right, but once you decide that it is going to be k -th order Markov then you are stuck with keeping a k -window history around and since you are treating this k -window history as your state you are unnecessarily relearning the same thing over and over again so there might be the first five states might be irrelevant.

But every time I visit a different set of states in the beginning I will treat it as a different state a different, a different observation state, right so these things mean wastage of computation that is what I meant, right. So if you can fix it that will be great right, there is also another problem with using k -th order Markov why, what is the other problem determining the k so you might actually get the k wrong in which case either you will be doing wasteful computation.

Because you have too large a ' k ' or you will be missing out on important information because you chose a too small a ' k ' okay, so these are all the problems with these history based methods but history based methods are very easy to use, right you do not have to worry about too many complicated things for example you do not have to worry about a belief state, right we already saw updating the belief state is a very complex operation, right.

So we needed to worry about how would you do this updation things like that we will talk about that depending on how far I make it today, right so we will talk about that and right, so it is just a lot easier to implement intuitively easy to understand and if your MDP happens to have a small k this is k -th order markov with a small k right, these are pretty good at solving the problem you can go ahead and do that.

So there is one method which I am not going to go into in detail I encourage you to read no, I will not ask questions on this in the exam it is called the U trees, right the U trees are proposed by Andrew McCullum when was this long time ago okay, I forget 92 maybe yeah, so what he does in U trees it is U tree stands for utile trees so it basically he only makes distinctions in the history, so he treats all the states as just one thing to begin with, right so just a single state your problem does not have multiple states it just has a single state.

And then as you make more observations and you get more rewards he only makes distinctions okay, that allows you to make better predictions of the rewards you are going to get, okay. So I will not worry about everything else I will only worry about the first bit I see okay, so if the first bit is 0 or 1 should I make a distinction based on that right, so I split it into two states in one state the first bit is 0, in the other state the first bit is 1, right so does it make sense does it give me better prediction if it does okay, great.

Now should I look at the second bit now or should I look at the first bit one step ago right, so there are multiple things I can I can look at more bits in the current state itself or I can start looking at history, right okay I am mixing up two things I am mixing up a little later work so what Andrew did was the following do I look at one step down, okay so what is the first state so last state was 1001, right I don't need to look further back in history, I am done because any further back in history I look I am not able to make any distinction between the predictions, because as soon as I know that the last state was 1001, I know exactly what will be the outcome of each of those actions.

So I do not need any further history, but then I look back and see that the last state was 1010 right now I need to look further one more step back, right so I have to look at 1010 and then I look one more step back if it is 1001 I am done, right if I look one more step back and it is also 1010 ok then maybe I have to look back one more step, right or look forward so I had looked back one more step right.

So like that so we will have to look at what the history is see what is it that you observe and based on that you have to look further and further back in the history, right so this is essentially

what he what he did so he did not say that I will always use a history of length 3 I will always use a history of length 4 what length history I use depends on what are the actual elements that go into the history, I did not tell you that U trees was data efficient or memory efficient it is not it is in fact it requires a horrendously large amount of memory, in fact U trees as it was originally proposed by McCullum is also very wasteful of data, right.

He wanted to give some guarantees about, statistical independence and so on so forth so what he essentially did was whenever he makes a split right he kind, of throws away all the data, and then he starts all over again okay so that is pretty wasteful use of data right, so it is not I am not using the same trajectory to determine multiple splits that gives you independence in the tests right but also wasteful of data so we need to come up with other mechanism further U trees is amazingly powerful right he actually has solved nontrivial problems which the other methods for partial observability would struggle with in this because given the kind of state spaces is looking at right.

But he does do a very good job with that so that is basically on history based methods so the second class of approaches, which I would call this is the Q-MDP like approach I cannot think of an appropriate term for it but Q-MDP is a prototypical example of such kinds of models is that you assume that the MDP is available to you yeah you assume that you know the MDP okay since you know the MDP what you do is go ahead and solve it, you solve the MDP get a policy then what is the big deal, you assume that you know MDP is known.

So you have this POMDP model right I assume that my S A P R is known, I will solve the problem I can solve do value iteration policy iteration whatever you want solve it I get my policy π_i , now what happens, exactly so execution becomes a problem, right so I can solve execution is still a problem so can you think of ways of fixing that, yes so I have the POMDP I have the have the POMDP I have let us say I have the POMDP.

So I solve the MDP component of it now when I have to execute what do I do, because I learnt the policy on the states how can I do it on the observations, ok even if I want to do it on the observations, what should I do, no we are not I said we have the POMDP I take the MDP

component of the POMDP and I solve it, so I have a policy defined on the states, right in the history based methods we are assuming you only have these observations as states, right in the Q-MDP we are assuming you are given a POMDP you have to solve it right.

So we solve it we find the policy because I just take the MDP component out and I find the policy now, how do I execute it let's say I found the Q function that is why it is a Q-MDP ok I have somehow I ran sarsa I ran Q-learning or I ran value iteration on the Q function right I ran policy iteration with the Q function whatever it is I have the Q star for the MDP exactly there I have O, belief somebody said belief exactly.

So what do you do with the belief yeah yeah you use belief yeah belief of s into ..yeah Q-MDP use the Q function use the Q function so essentially what you do is you is the some kind of score for right and then the action that you pick would be so you are in the current belief state right or you can think of this as so at the current belief state,

So what is the value of that for taking action a in the current belief state current belief state the value of taking action a is essentially given by the sum over all states, belief state belief of being in the state into Q s a and then when you take an action you basically do this right so, now I have solved the Q right and converted that into a value function over belief states comma actions so we will look at the other popular method of solving partially observable MDPs, right.

Unfortunately all the methods that are out there except, history based methods assume that you know the POMDP right or you take a Bayesian approach to solving this where you start off with some distribution over all parameters of the POMDPs and then you try to solve using that but in whatever it is you have some kind of models, that you are using right.

So given that you are anyway having a model anyway the assumption you make is that you know the model while you are solving POMDPs this is a very easy way of solving it you just solve MDP first and do this what is the problem with this method though, you have to maintain belief over all states for all belief state methods, so that is the only non belief state thing everything else we have we will talk about we will be maintaining belief states, we need the, no no that is what I am saying right.

If you do not have the MDP you do not know anything about it that is your only option when you are executing only one policy yeah but you are getting somewhere close, no no, I have learned it right do value iteration do policy iteration do whatever you have a MDP you know how to solve it, yeah that is there for any belief state method, since we solve the problem assuming that you would have access to the state at that point.

And now you are only doing a heuristic for converting that into an execution policy, the policy that we execute might not be optimal for the POMDP if you solve the POMDP directly then you can find something that is optimal for that POMDP, right given now the solution you have discovered is assuming that you would know the state at all points right. So given the uncertainty in the states, there might have been a better action to pick in terms of the total reward that you get I mean if I had for example right here is a very dumb way of thinking about right which is actually not too dumb, later as you will see.

I can take my POMDP, right I have this way of computing the belief right so we will come to that but we you have a gist of how to compute the belief now I can define an MDP where my states are the belief states right, that function will give me the transition probabilities right so given that this is the initial belief state and this is the observation that you made right given this is the action you took or something what will be the next belief state in fact that will become actually almost deterministic right because it is actually a computation that will determine what the next state is, so I will have a belief state I will do some computation I will get to the next belief state right now it is an MDP, right I know the previous state right and then if I know the previous belief state my current action will determine what my next belief state will be if I know the previous belief state comma the observation right if I know the current action I know what the next belief state will be

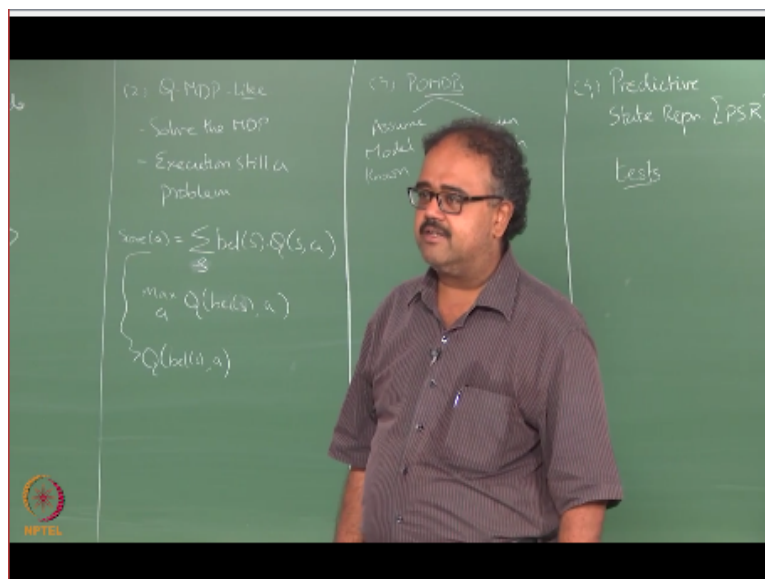
So now it becomes an MDP I can solve it right what is the problem with solving that MDP, I have a very high dimensional, continuous state space, right if I original originally I had like a million states now my state is a million dimensional simplex right so it is like a crazy state space, and I have to learn value functions on top of that, right so it becomes a little painful so what we

will see in the I am going to defer it to the next class I do not have actually energy to teach the whole of POMDPs today.

So what we will see in the next class is how to solve the belief state MDP in in a clever fashion, by making use of certain kinds of regularities, that will be available in the value function of belief state MDPs, the very fact that the belief states are simplices right and the belief state is a simplex and then the value functions have to be related in a certain way.

So we will use those relationships and try to come up with efficient ways of solving POMDPs directly where you are essentially learning an action from history, learning a policy from history to actions, right so that is essentially what you will do, when we solve POMDPs right so such solutions where you are actually cognizant of the entire history of what has happened right and then making choices and knowing that knowing that there is a certain uncertainty in where you are and being cognizant of that and making your action choices certainly yields better policies then something ad-hoc like this, okay what is nice about the ad-hoc thing it is easy, it is much easier to implement and, turns out that in practice it actually yields good policies, they are not too bad.

(Refer Slide Time: 34: 32)



Right so people have tried in most real problems that you encounter, right so things are not so pathological that this will perform very badly, in many real problem domains that you encounter, Q-MDPs are seem to give you good enough solutions so people actually like to use this only problem is you cannot prove anything, right so the third class of methods are directly solving directly solving POMDPs, so it requires setting up a lot of machinery for that so we will come to that later but there are two, two classes of algorithms here.

So one assume that assume the model other one is a other one is like a Bayesian approach where you do not assume the model right but you assume some kind of distribution over all possible models, right and then as you gather more data you try to reduce that distribution also but at every point of time you can actually solve a POMDP even though you do not know the true model right you can solve some assumed model and use that to generate some kind of experience, and use that experience to refine your probability distribution over the models, okay.

So you could assume the model and solve it assume that you know the model and solve it or you can take a Bayesian approach of course you can take a non Bayesian approach also when the model is not known right you can just do some kind of a frequentist approach just count the number of times some things have happened and so on so forth, but this is more fun. Ok, so the last class of algorithms which I will not cover a lot right but there is a video for you guys to look at, can you ask them to link the PSR videos something called predictive state representations this kind of turned the whole idea of history based methods on its head so what did they do they said why am I bothered about history.

So what do I need the history for, so that I can make better predictions about the future right so instead of worrying about history, can I just look at how well can I predict the future right so what are the things that you need to know about the future right so that I can predict everything that can happen in the future, so the basic assumption here is if you make certain assumptions about the dynamics of the world right I can do the following I can say that if I knew what is the probability, of let us say if I go forward two steps I will get a wall from wherever I am if I go

forward two steps I will get a wall if I go to the left five steps I will get a wall, if I go to the right two steps and go up one step I will get a wall.

So if I know all of these ok what is the probability of these four things happening then I know where I am in the world, kind of makes sense right instead of trying to know about the history if I can make predictions about where I will end up at if I if I do some small policy fragment right and I will make certain observations along the way right so essentially if I say after three steps I will hit a wall that essentially means one step no wall two step no wall three step wall right.

So it is a sequence of action observations that you decide that you will have to decide based on the problem dynamics, so they have mechanisms for coming up with the tests themselves right so these things are called tests so what is the probability of North clear North clear north wall okay that is one thing right ok that is one sequence so action North, observation clear action North observation clear action North observation wall that is one sequence likewise action west observation wall that is one sequence then action East no wall action east no wall action North one wall right that is another sequence, right these kinds of observation action sequences are called tests, what they did was they said that predictive state representation consists of a set of tests right and the probabilities that those tests will be true, right.

So if I have this collection of tests and their probabilities that will determine what a state is for me that is why it is called predictive state representation, right so what is a nice thing about it is they do not assume that there is an underlying MDP like the POMDP does right they do not ever want to look at the MDP it is like saying that okay I do not need all the history sequences, I will need only some of the histories and that is enough to represent the world for me right.

So in predictive state representations we do not assume that there is actually an underlying state space we only have these tests right, and we say these tests are adequate for me to make any decisions I want in the world there might be a true underlying state we do not know, right but that is a very powerful thing about PSR, if you build PSRs on top of POMDPs then you can show that they are as powerful as the direct POMDP solution methods like they yield the same solutions and so on so forth,. but the nice thing is PSRs can go way beyond the regular POMDP

setting because they do not have to have any notion of what the underlying state space is they can have they do not need a state space see the video.

So this was proposed by Satinder Singh and his group and the video is by Satinder Singh so you are getting it from the horse's mouth, so Satinder came down here for the reinforcement learning workshop, so I asked him to do a session on PSRs he did so that makes it that means that I do not have to ever teach PSRs again right so that is a cool idea, unfortunately it is hard to learn the PSRs right that is, it is so, history based methods it is easy just look back so many time steps and you just remember whatever you wanted to learn but learning the PSR turns out to be a little tricky you can learn linear PSRs right.

So what I mean by linear PSRs, linear PSRs are a set of tests, from which I can recover the probability of anything else being true, right suppose I said I gave you three examples right so north, north, north and then east and then west, west north or whatever right I gave you three examples suppose using these three things I can give you the probability of west clear west clear west wall, by using a linear combination of these probabilities then they are called linear PSRs right and linear PSRs will necessarily be large, they will be if there is an underlying POMDP if there is an underlying MDP on which this PSR is defined, the number of linear PSR linear tests, that you need right will be the same as the number of states, the tests that you need will be the same as the number of states, in a linear PSR.

But if you are able to define nonlinear relations that can maybe products and divide one by the other and those kinds of things right you can even get a representation which is smaller than the number of states, right but then finding non-linear PSRs is incredibly hard we only have like existential proofs but actually finding these things are very hard, right so there is this whole literature on predictive state representation, is very interesting.

If there is interest next semester if people are around we could think of looking at PSRs if there is interest but I am more personally inclined to look at more of the deep RL stuff but we will see so we will but I am not going to cover this in more detail so if you want you should look at the

lectures so next class what is left is I am going to talk about POMDPs right so we will just do that in more detail and that will be the end of it.

IIT Madras Production

**Funded by
Department of Higher Education
Ministry of Human Resources Development
Government of India**

www.nptel.ac.in

Copyrights Reserved